Project: House Price Prediction

Dataset Used: kc_house_data.csv

This document outlines a structured, complete report of project based on training the module, predicting the test data and evaluating the accuracy on given test data.

Project Goal: Train a module to predict the classes using kc_house_data.csv dataset.

The primary libraries/modules required for this project are:

- **Pandas**: For handling datasets.
- **NumPy**: For numerical operations.
- **sklearn**: For statistical machine learning.

Project Coverage:

Category	Skills Covered
Data Pre-Processing	Data Loading.
Create Learning Dataset	Select the required columns for passing as Input.
Training the module	Algorithms to train the module using the dataset.
Predict the result	Algorithms to predict the species class

In the code we are importing the required libraries to load the dataset and creating a training set to train the module. After training using the dataset, we are using different classes from the sklearn module to predict the species class with different algorithms as shown below.

This score tells us how well the model fits the data—the closer the number is to **1.00**, the more accurate the model is.

1. Multiple Linear Regression

Linear Regression models the relationship between features and price by fitting a linear equation (a straight line) to the data.

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this code, we import the LinearRegression class and use the fit method to train the **model**. Then, we use the model to **predict** values for the test set and print the **R^2 score** to see how well it performed.

2. Decision Tree Regression

This model breaks down the dataset into smaller subsets while simultaneously developing an associated decision tree (flowchart-like).

```
from sklearn.tree import DecisionTreeRegressor
regressor = DecisionTreeRegressor(random_state=0)
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this code, we import the DecisionTreeRegressor class and use the fit method to train the **model**. We then use it to **predict** results for the test set and print the **R^2 score** to evaluate its performance.

3. Random Forest Regression

An ensemble method that operates by constructing many decision trees and outputting the average prediction of the individual trees.

```
from sklearn.ensemble import RandomForestRegressor
regressor = RandomForestRegressor(n_estimators=100, random_state=0)
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this code, we import the RandomForestRegressor class and use the fit method to train the **model** using multiple trees. Then, we **predict** the test set values and print the **R^2 score** to check the accuracy of the ensemble.

4. Support Vector Regression (SVR)

SVR tries to fit the error within a certain threshold (epsilon) instead of just minimizing the error, using kernels for non-linear data.

```
from sklearn.svm import SVR
regressor = SVR(kernel='rbf')
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this code, we import the SVR class and train the **model** using the fit method. We then use the model to **predict** the outcomes and print the **R^2 score** to see how well the support vectors fit the data.

5. K-Nearest Neighbors (KNN)

KNN predicts the value of a data point based on how similar it is to its surrounding neighbors in the training set.

```
from sklearn.neighbors import KNeighborsRegressor
regressor = KNeighborsRegressor(n_neighbors=5)
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this code, we import the KNeighborsRegressor class and train the **model** using the fit method. Afterward, we **predict** values based on the nearest data points and print the **R^2 score** to measure performance.

6. Gradient Boosting Regression

This builds the model in stages; it adds new trees that focus specifically on correcting the errors made by previous trees.

```
from sklearn.ensemble import GradientBoostingRegressor
regressor = GradientBoostingRegressor(n_estimators=100, random_state=0)
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this code, we import the GradientBoostingRegressor class and train the **model** sequentially using the fit method. We then **predict** the test set results and print the **R^2 score** to see how well it corrected errors from previous trees.

7. Ridge Regression

A type of linear regression that uses "L2 Regularization" to prevent overfitting by penalizing large coefficients.

```
from sklearn.linear_model import Ridge
regressor = Ridge(alpha=1.0)
print(regressor)

regressor.fit(X_train, y_train)
y_pred = regressor.predict(X_test)

print('\n'+'-'*20+'R2 Score on the Test set'+'-'*20)
print("{:.2f}".format(r2_score(y_test, y_pred)))
```

In this code, we import the Ridge class and use the fit method to train the **model** with regularization. Finally, we **predict** the test set values and print the **R^2 score** to evaluate how well it handled the data coefficients.

8. Lasso Regression

Uses "L1 Regularization" which can shrink some coefficients to zero, effectively performing automatic feature selection.

```
from sklearn.linear_model import Lasso
model = Lasso()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('-'*80, "\nLasso Regression Performance:")
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

In this code, we import the Lasso class and use the fit method to apply L1 regularization. Finally, we predict the test set values and check the metrics to see how well it performed automatic feature selection.

9. AdaBoost Regressor

"Adaptive Boosting" that weights difficult-to-predict instances more heavily in subsequent iterations.

```
from sklearn.ensemble import AdaBoostRegressor
model = AdaBoostRegressor()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('-'*80, "\nAdaBoost Regressor Performance:")
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

In this code, we import the AdaBoostRegressor class and use the fit method to boost weak learners. Finally, we predict the test set values to see how it handled difficult-to-predict instances.

10. Bayesian Ridge Regression

A probabilistic approach to linear regression that includes regularization parameters within the estimation process.

```
from sklearn.linear_model import BayesianRidge
model = BayesianRidge()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('-'*80, "\nBayesian Ridge Performance:")
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

In this code, we import the BayesianRidge class and use the fit method to apply probabilistic estimation. Finally, we predict the test set values to evaluate the stability of its predictions.

11. Extra Trees Regressor

"Extremely Randomized Trees" that choose split points at random to reduce variance and improve training speed.

```
from sklearn.ensemble import ExtraTreesRegressor
model = ExtraTreesRegressor()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('-'*80, "\nExtra Trees Performance:")
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

In this code, we import the ExtraTreesRegressor class and use the fit method with extremely randomized splits. Finally, we predict the test set values to evaluate its performance against standard random forests.

12. Huber Regressor

A regression model that is robust to outliers; it uses a linear loss for samples classified as outliers and a squared loss for samples classified as inliers.

```
from sklearn.linear_model import HuberRegressor
model = HuberRegressor()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('-'*80, "\nHuber Regressor Performance:")
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

In this code, we import the HuberRegressor class and use the fit method to minimize the impact of outliers. Finally, we predict the test set values to evaluate its robustness compared to standard linear methods.

Conclusion:

In this project from the given dataset we can trained different models with different algorithms to predict price of the house. We can also see the accuracy of the algorithms that are used to train the models. Once we pass the required values for the input the code will predict the price of the house and provide us with the price with more accuracy.

What you have learned:

In this project we learnt that to predict the price of house we can use different algorithms to train different models and predict the price with more accuracy. And how we can use the most suitable algorithm to train a model depending on the requirement.